

Gestión Lean-Agil de Productos de Software

Manifiesto Ágil: documento redactado en 2001 por expertos que provee la filosofía del desarrollo ágil

PRINCIPALES MANIFIESTOS → VALORES

Individuos en interacción por sobre procesos y herramientas
(mejor un equipo motivado y con buen funcionamiento que herramientas y procesos excelentes)

el desarrollo de software es una act. Humana

Software funcionando sobre documentación extensiva
(mejor software estable que documentar todo)
con valor al cliente

solo documentar lo que agrega valor al cliente

Colaboración con el cliente sobre negociación contractual
(mejor involucrar al cliente en el producto para que dé buen feedback que el seguir un contrato inmutable)

Responder a cambios por sobre seguir un plan
(mejor adaptarse a cualquier cambio y dar siempre valor que seguir un plan rígido)

Filosofía Ágil → el objetivo primordial es entregar software de forma frecuente en un entorno cambiante. Esto se usa en entornos de gran variabilidad y se involucra mucho al cliente en el proceso y que de retroalimentación

Adopta un ciclo de vida iterativo-incremental donde el Empirismo es lo primordial

Pilares Fundamentales del Empirismo

Transparencia

nos permite crecer como equipo
cada uno aporta y hace que el conocimiento sea explícito)

Inspección

(permite detectar variaciones o problemas. es el momento de aprender del error)

Adaptación

(la inspección permite adaptación en función de los resultados cambiando lo que se cree mejor. Hacemos ajustes del proceso)

Requerimientos en Ambientes Ágiles - VS.

- Usar el valor para construir el producto correcto (foco en dar valor al negocio)
- Determinar que es solo suficiente (los req van a ir apareciendo, no inventar)
- Usar historias y modelos para mostrar que construir (trabajar con un referente del lado del negocio)

Vínculo proceso-proyecto-producto en la gestión de un proyecto de desarrollo de software



El proceso que se selecciona se adapta al Proyecto el cual se incorporan Personas y se obtiene como resultado un producto. Se usan Herramientas para automatizar el proceso.

Dato de plata $\Rightarrow$ soluciones que podrá hacer frente a la crisis de software (NO EXISTE)
y los problemas de productividad o complejidad.

Dificultades de desarrollar sistemas

Esenciales

(todo aquello que trae el construir software)

• Problemas Incalculables

Complejidad $\Rightarrow$ crecen tamaño es sofware
crece exponencialmente

Conformidad $\Rightarrow$ debe realizar lo que el cliente quiere además a los req.

Volubilidad $\Rightarrow$ el sofware está sujeto a cambios y evoluciones

Invisibilidad $\Rightarrow$ dificultad con representarlo
se usan los grafos que es bueno

Accidentales

(aspectos que dificultan la construcción pero
no han sido solucionados a lo largo del tiempo)

• Falta de Automatización y escalas

• Lenguajes de programación de alto nivel

• Tiempo compartido

• Entornos de programación unificados

en el enfoque ágil la triple restricción lo que varía es el Alcance y está más fijo el tiempo y el costo. Ya que está dirigido por el VALOR

Big Requirements Up Front

20% se usa casi siempre
45% no se usa nunca
nunca se conoce el 100%

Product Backlog

Lista priorizada de características y requerimientos del software.
el POwner se encarga de definir la lista y la priorización.

A diferencia del enfoque tradicional que los req. se definen todos al inicio el enfoque ágil el product Backlog nunca está al 100% siempre salen req. nuevos incluso al final del proceso. Estos req. se obtienen de la comunicación cara a cara con el cliente. (No se escribe en ERS) simplemente se le pregunta al cliente.

Tipos de Requerimientos

Req. Negocio (valor al negocio)

Req. Usuario (medio por el que vemos a lograr el valor)

User Story →

no sirve para req. software, sirve para req. users

Req. Funcional

Req. No Funcional

Req. de Implementación

Principios relacionados a Req. Ágiles

1- prioridad satisfacer al cliente

2- reducir cambios de req. incluso en etapas finales

4- técnicos y no técnicos trabajar juntos

6- medio de comunicación cara a cara

11- equipo autoorganizado

User Stories

Técnica para trabajar req. de usuarios → Descripción corta de una funcionalidad que se espere de un usuario del sistema.

Son escritos por el P.Owner

- Representan necesidad de usuario, de forma modelable
- Conforman una descripción del producto
- Son utilizados como ítems de planificación
- Se usa como recordatorio en la conversación
- Junto con el código sirve para documentar

DoR (Definition of Ready)

medida de calidad que determina si la US está en condiciones de entrar a la iteración de desarrollo

- 1- Cumple con INVEST Model
- 2- Pueden el PO o el equipo agregar cosas extras.

INVEST Model:

Independiente (como se implem en cualquier orden no debe haber relación con otras US)

Negociable (no es un contrato estricto sino para una guía de lo que quiere el PO y lo que se va a construir)

Valuable (deben aportar valor al negocio y poder definirlo)

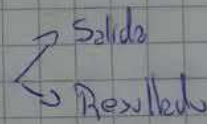
Estimable (debe poder ser estimable (binario, complejidad, esfuerzo, incertidumbre) ver si entra en una iteración)

Small (debe ser pequeña, que se pueda terminar en una iteración)

Testable (debe cumplir con los pruebas de aceptación para ver si se implementa bien)

DoD (Definition of Done)

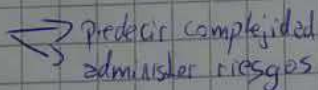
medir cuando una US está lista para presentar al cliente



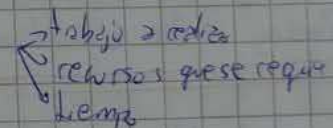
Estimaciones

proceso de obtener una aprox sobre una medida con el objetivo de predecir la complejidad y gestionar los riesgos en un proyecto

estimamos para



se estima



Las estimaciones se van recalculando ya que al inicio no tenemos suficiente info por lo que puede diferir mucho.

Métodos

→ Basados en la experiencia

- Datos históricos (en base a información de proyectos similares) no se usa mucho en agil.
- Juicio experto

- Juicio experto puro (un experto estudia el caso y hace la estimación ya que tiene experiencia. Si se va de la org se pierde)
 - (optimista, pesimista, histórico) y se hace una fórmula
- Juicio Wideband Delphi (grupo de personas se junta para converger en una estimación. Coordinador jala todo y se quita memoria)

Partes:

- ① **Tarjeta** (descripción de la historia para planificar como desarrollarla
(nombre rol - actividad - valor negocio - frase verbal - criterios de aceptación - pruebas de aceptación))
- ② **Conversación** (lo más importante de la US, explica la comunicación entre POwner y el Equipo)
→ discusiones acerca de la historia
- ③ **Confirmación** (definición entre el PO y el equipo para demostrar que la funcionalidad del producto se implementa)
conforman la definición de los pruebas de aceptación

Ventajas

- Priorizar comunicación verbal sobre la escrita
- Comprensibles para el PO y el equipo
- Tamaño para poder planificar

Desventajas

- Proyectos grandes difícil ver la relación entre US
- No es lo ideal para equipos grandes

Product Backlog

es una pila de (Product Backlog Items) con las estimaciones de tamaño. En lo alto están las US de mayor prioridad y granularidad fina y abajo las de menor prioridad. (menos detalles)

se agregan o se borran nuevas US en cualquier momento

Las US son porciones verticales → para agregar valor deben tener un poco de cada capa (UI/BD/Back)

Las US son preferibles que sean pequeñas y muchas que pocas y grandes

→ Para determinar el tamaño

Story Points

→ según se coloca un número

Complejidad	(dificultad de la funcionalidad a hacer)
Esfuerzo	(tiempo en horas para implementarla)
Interdependencia	(técnica: no sabemos del dominio de solución, tecnología o herramientas) (de negocio: no sabemos como va a interactuar el user)

No importa el valor en sí sino la comparación.

(se usa Fibonacci)

Nivel de Abstracción

- **US** cumple con el criterio Listo (DoR) para ingresar a la iteración
- **Epica** US grande que no entra en una iteración → hay que dividirla (no cumple INVEST)
- **Tema** conjunto de US relacionadas. Se las agrupa para poder de estimar o planificar de forma más rápida

Consideraciones

- se debe tener en cuenta el criterio Done
- el tiempo para realizar las estimaciones time boxing

Framework SCRUM

-
-
-

Herramientas SCRUM

- Tarjeta de tarea

• Tablero (Taskboard)	story (nombre de US)	To Do (tareas a realizar)	In Process (tareas realizando)	To Verify (tareas a verificar)	Done (tareas hechas)
-----------------------	----------------------	---------------------------	--------------------------------	--------------------------------	----------------------

- Gráficos de Burdy

Sprint Burn-Down Chart

(cuanto trabajo queda por terminar) en puntos de historia



Sprint Burn-Up Chart

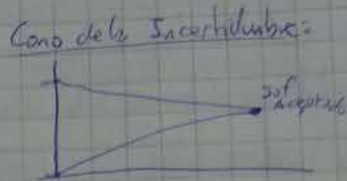
(cuanto trabajo se ha completado a lo largo de los sprints)



Métricas

- Proceso (mismos que el proyecto pero enfocado en el conjunto de proyectos realizados)
- Proyecto (velocidad del equipo, esfuerzo realizado, eficiencia de estimación tiempo por tarea)
- Producto (cant líneas de código, cant métodos, velocidad de respuesta)

- Basado en reviews
- Basados en el mercado
- Métodos algorítmicos



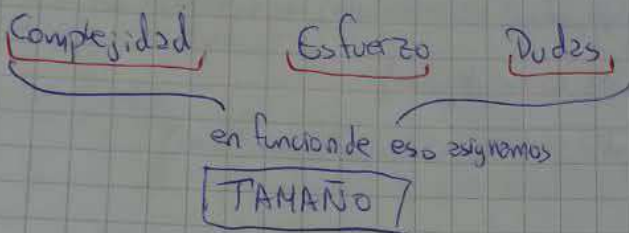
Estimaciones Ágiles

Quien estima es el que realiza el trabajo (el equipo de trabajo menos el PO)

Lo hace más certero ya que el equipo puede debatir y llegar a conclusiones más certeras.

- Medidas relativas no absolutas.
- No es una medida basada en el tiempo
- Comparar lo hacemos mejor

Estimamos para → ver cuantos US. entran en una iteración
→ ver cuanto es la velocidad del equipo



Story Points

unidad de estimación que especifica que la complejidad, incertidumbre y esfuerzo tiene el equipo respecto a otro US.

Velocidad → que cant de Story Point correspondientes a los US fueron aprobados por el PO Owner en una iteración.

↓
medida de estimación

Duración del proyecto = cant Story Points de todos los Users Stories / la velocidad del equipo.

Poker Planning o Poker Estimation

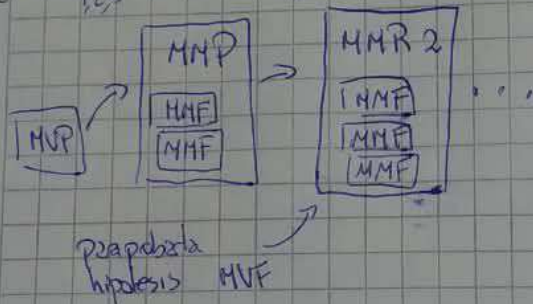
- Conjugación de diferentes métodos como (juicio experto, analogía etc). En la cual se usa la serie de Fibonacci para estimar ya que se dice que la complejidad del software aumenta exponencialmente.

Procedimiento:

- 1- se listan US
- 2- se acuerda una US con el menor grado de incertidumbre y se la define como Canónica (1 o 2 points)
- 3- cada miembro elige una carta con el valor que cree que es el tamaño de la US teniendo en cuenta los 3 aspectos y la US canónica.
- 4- se dan vuelta las cartas. Si son todas iguales se coloca ese número como estimación. Si no el que dio el valor más bajo y más alto exponen y lo justifican. Los otros también opinan.
- 5- se repite el paso 3, y así hasta que se llegue a un acuerdo.

MRP (Minimum Marketable Release)

- (MMF) (característica mínima comercial) pequeña pieza de funcionalidad que ya puede ser liberada
- (MMP) primer release dirigido a los primeros usuarios tiene los MMF esenciales
- (MMR) Release de producto con las características mínimas



El objetivo es llegar al MLP (mínimo producto memorable) ya que actualmente lo viable no es suficiente al haber tanta competencia. Es necesario crear una conexión con el cliente

Gestión de Software como Producto (gestión de configuración) → SCM

estimación tradicional:
tiempo, esfuerzo, atenderlo y costo

} tiempo por complejidad

estimación ágil:

tiempo, tiempo (duración), costo
 ↓
 story point
 se deriva del
 cálculo de la velocidad del equipo
 → P por sprint → en SP

va cambiando la estimación
 cada vez (iteración)

según el nuevo
 esfuerzo

capacidad → velocidad estimada

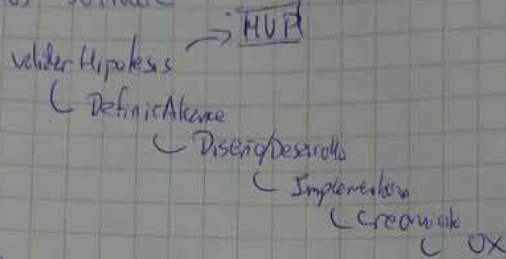
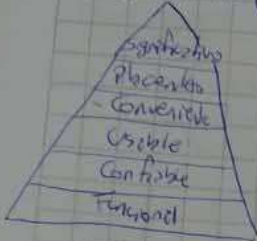
puede ser en SP
 ↓
historial de trabajo

Métricas Ágiles

- Velocidad (cant Puntos de historia por sprint que fueron completados)
- capacidad (velocidad estimada $\rightarrow$ compromiso de un equipo por un sprint.)
- RTF (cant features testados y funcionalidad)



Gestión de productos software



Focalizado
Tareas

Focalizado
Experiencias

MVP (mínimo producto viable): versión de un nuevo producto que permite al equipo recopilar la cant. (tiene que haber todas las cosas) máxima de aprendizaje validado sobre los clientes con el menor esfuerzo.

- 1- Tiene el valor suficiente para que las personas estén dispuestas a usarlo
- 2- Demuestran beneficios a futuro para retener a los usuarios
- 3- Proporcionan un ciclo de retroalimentación para guiar el futuro desarrollo.

Permite validar Hipótesis:

Hip. de valor: si realmente está entregando valor al cliente luego de usarlo (tasa de retención $\rightarrow$ que use el producto nuevamente)

Hip. de crecimiento: cómo nuevos clientes descubren el producto. Ver si las personas desean usarlo.

① Un producto tiene una hipótesis de valor único (UVP)



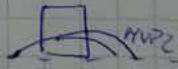
② Crear un producto mínimo viable y probar su hipótesis. Se centra (MVP) en su propuesta de valor pero con un poco de funciones para asegurarse que sea viable (testables)



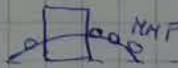
③ El MVP también es una hipótesis y hay que validar que tenga un mercado para el producto



④ Si ves que las respuestas apuntan a una modificación se revisa (MVP2) la hipótesis y construyes otro MVP basado en la nueva hipótesis



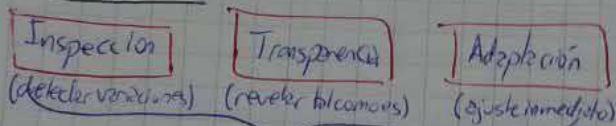
⑤ El nuevo MVP es un éxito y así necesitas más crecimiento y ves que hay áreas que al menos pueden traer este crecimiento. Características mínimas comercializables (MMF)



⑥ En caso de que estas áreas tengan alto potencial pero alta incertidumbre se enfrenta con MMF (características mínimas viables) si no resulta exitosa se cambia hacia otra área. Si es exitosa se desarrollan más MMF (versión mini del MVP)



Empirismo



Empirismo

Manifesto Ágil → conjunto de valores y principios

- | | |
|-------------------------------|------------------------------------|
| (4) | (12) |
| interacciones - | 1. satisfacer al cliente |
| software funcional - | 2. aceptar cambios |
| colaboración con el cliente - | 3. entregas frecuentes funcionando |
| responder al cambio | 4. trabajar juntos |
| | 5. individuos motivados |
| | 6. conversación cara a cara |
| | 7. exhibir software funcionando |
| | 8. promover desarrollo sostenible |
| | 9. buen diseño, atención técnica |
| | 10. simplicidad esencial |
| | 11. equipos auto-organizados |
| | 12. ajustar comportamiento |

✓ Scrum: El Framework (marco de trabajo)

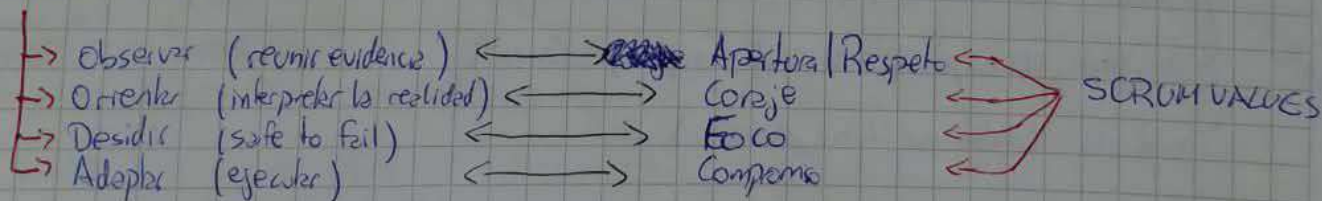
Marco de trabajo ligero que ayuda a las personas, equipos y org a generar valor a través de soluciones adaptativas para problemas complejos

≠ metodología (no es un peso a peso a seguir)

Principios:

- Control de proceso empírico
- Auto-organización
- Colaboración (entre todos los roles)
- Priorización basada en valor (backlog priorizado)
- Time-boxing (los eventos se deben ejercer en un rango de tiempo) → Predefinibilidad
- Desarrollo iterativo (ciclos de trabajo iterativos-incremental)

* Ciclo OODA



Team Autogestionado

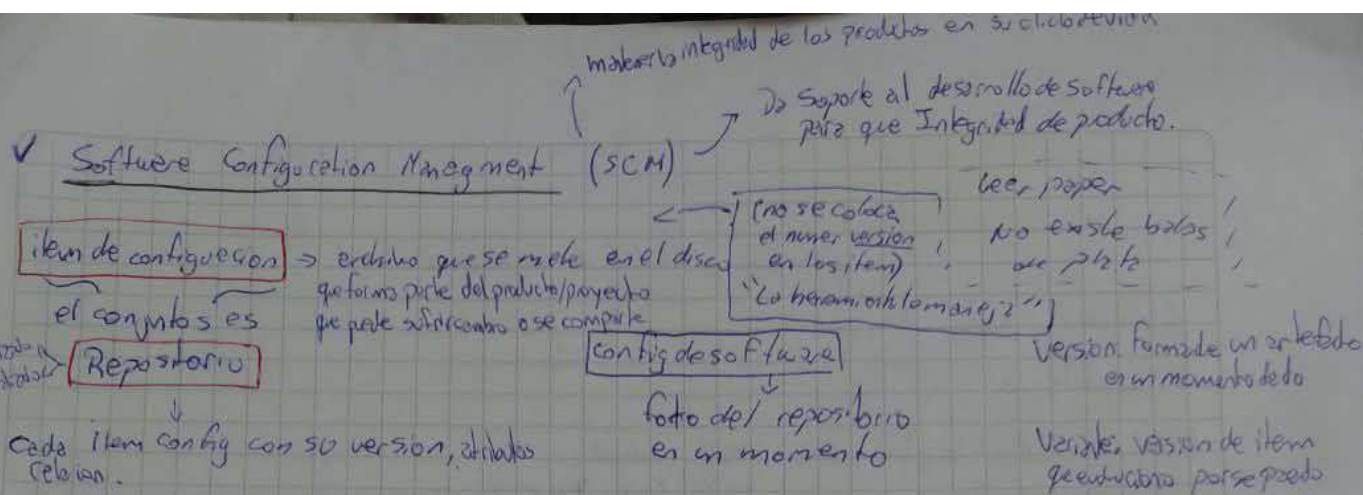
- El equipo decide todo
- Responsabilidad total
- Diversidad cognitiva

* Ecosistema extendido: Stakeholders - Supporters

Clients, users que proveen feedback y validan el valor

Ciudades eliminan barreras organizacionales proveen recursos





Incorporar para config:

- Identificar en un momento dado
- Controlar sistemáticamente sus cambios
- Mantener su integridad y origen

Que significa integridad del producto

- satisface las necesidades de las com.

Actividades fundamentales:

1- Identificar item de config.

Definir reglas de nombrado/ubicación/estructura Repo.

Nombre	Regla	Ubicación	Tipo
Clase Cod Fuente	AVNombreClase.java	.../AV/CodFuente/	subsisistema/AV/

Plan de Software Configuration Management

Item de config que (SCM)

(parámetros, reglas de nombrado, estructura de repo, etc)

2- Control de cambios

Comité de control de cambios (CCC)

Línea Base (configuración que cumple con las pruebas y requerimientos)

checkPoint

Configuración → todos los item de config en un momento dado

Tipos LB

- Operacionales (tienen código)
- Especificación (otros item de config documentación) (Primer LB)

1.2 + 1.0 Version

Remas

3- Reporte de estados

Trazabilidad de cambios

Física (ver si el plan de proyecto está en el repo, validar los items)

4- Auditorías

de config (Proceso de evolución objetivo) es Externo

Funcional (ver la integridad y ver lo que está fue la trazabilidad está bien)

Herramientas

Matriz de trazabilidad

este req → hizo estos archivos → con estos pruebas

CheckList

List con preguntas

se audita a 1 Línea Base no al repo del proyecto

Roles

- Product Owner	Humano
- Product Developers	Humano/IA
- Scrum Master	Humano
- Stakeholders	Humano/IA
- Supporter	Humano

(maximizar valor largo plazo, gestionar product Backlog)
 (crean el incremento)
 (gestiona la efectividad y remove impediments)
 (entidades afectadas por el producto)

Genchi Genbutsu - liderazgo y cambio

↳ (Voy al lugar veo y escucho)

PO → el que? y por que?
 PD → el como? y la calidad
 SM → la efectividad

Gestiona Product Backlog
 Gestiona Sprint Backlog

Sprint (evento de longitud fija)

↳ timebox → (4 semanas max) → crear historias

↓
 mientras mas corto mas feedback (oportunidad para pivotar)

↳ inmutabilidad del objetivo → sprint goal inmutable

↓
 el alcance si es negotiable si hay aprendizaje y el goal no se modifica

Daily Scrum: 15 min

(replanificación diaria hacia el Goal)

"Sincronización técnica"

Sprint Review: 4hs

(sesión de trabajo colaborativa e inspección del resultado no solo calidad)

"validación de valor"

Sprint Retrospective: 3hs

(mejora del sistema y definición of Done)

"Foco en la calidad del equipo, procesos y herramientas"

Fin de Sprint

Sprint Planning: 8hs

(Diseño Técnico)

Definir → ¿Porque? El Sprint Goal → valor que vamos a
 ¿Que? Items seleccionados de Backlog
 ¿Como? Plan técnico a ejecutar

Timebox → tiempo asignado a un evento

requisitos de definición Done
 capacidad estimada

Cambio De Salida a Impulso

Output Done

entender de calidad técnica
 incremento utilizable
 construcción correcta?

Outcome Done

evidencia observable de valor
 cambio de comport. del user

¿Genera valor?

UNIDAD 1

Ingeniería de Software

- Disciplina que se encarga de todos los aspectos de la producción de software desde la primera etapa de especificación del sistema hasta el mantenimiento.
 - ↳ Producir de manera rápida y económica un sistema

➔ Crisis del Software:

en OTAN 1968. reduce la dificultad de generar software libre de defectos, fácilmente comprensible y que sea verificable.

CAUSAS:

- * La evolución del hardware fue muy grande mediante los circuitos integrados y permitió el desarrollo de sistemas más grandes y complejos. Pero el desarrollo de software no acompañó ese crecimiento ya que el software es una act. humana cuyo producto es abstracto e integral.
- * Demanda creciente de sistemas complejos y falta de una disciplina que intervenga en todas ^{las} etapas

Actualidad:

Demanda creciente debido a que se necesitan que se construyan y distribuyan más rápido y los métodos tradicionales de ing. no permite esa "espontaneidad"

Baja expectativa ya que muchas empresas no usan el método de la ing. softw. lo que hace más costoso y menos confiable y la empresa se conforma con que funcione y no con que le de un valor al negocio.

• Disciplinas que conforman la ing. softw.

Técnicas

(aportan al desarrollo como producto)

- Toma Req.
- Analisis
- Diseño
- Implementación
- Pruebas
- Despliegue
- Documentación
- Operación

Gestión

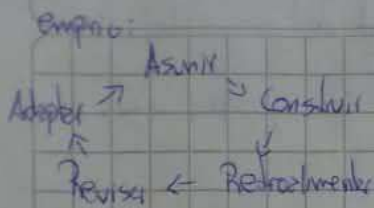
(act. de plan. ejecución, monitoreo control del proyecto)

- ej. LEAN, Agile
 - Planificar
 - Monitorear
 - Controlar

Soporte

(disciplinas transversales a los procesos que permiten verificar la ing y calidad de un producto)

- Gestión de configuración de software (scm)
- Métricas
- Aseguramiento de calidad (QA)



Componentes de un Proyecto de Sistemas de Información.

Proyecto de software, esfuerzo temporal que requiere del acuerdo de un conjunto de especialidades y recursos para la creación de un producto. Unidad organizativa para adeper el marco teórico del proceso (instancia)

se define el proceso, tareas, quien realizará cada una y mecanismos que se implementarán

Características:

- Único (caract. propias que lo diferencian de cualquier otro)
- Orientado a objetivos (objetivos $\rightarrow$ no ambiguo, medible, alcanzable) una vez que se llega se termina)
- Duración limitada (en principio y en fin y libera los recursos y personas que se usaron)
- Tareas interrelacionadas basadas en esfuerzo y recurso (se define previamente)

Administración del proyecto de software

Organización y coordinación de personas y recursos que permite desarrollar un producto según el presupuesto, tiempo y funcionalidades acordadas con el cliente. Asegurando alta calidad.



• Una buena gestión no garantiza el éxito del proyecto pero una mala gestión implica fallos en el proyecto

• Metas de la administración del proyecto

- 1 entrega al cliente en tiempo acordado
- 2 costos dentro del presupuesto
- 3 software que cumple con las especificaciones del cliente
- 4 equipo con buen funcionamiento y optimo

1. Causas de Fallos del proyecto

- fallo toma de Req (80%)
- fallo de finir problema
- planificar con datos insuficientes
- No hay seguimiento del Plan
- estimaciones basadas en supuestos y no en lo real.
- nadie a cargo

Ciclo de Vida (Modelos de Proceso)

serie de pasos a través de los cuales un producto o proyecto progresa en el tiempo
Representación simplificada de un proceso

- define elementos del proceso
- define flujos del proceso
- } define criterios para determinar si se debe pasar a la tarea siguiente

"El ciclo de vida de un producto es siempre mayor al del proyecto."

- el ^{c.v.} proyecto dura solo el desarrollo del software, en cambio, el ^{c.v.} producto dura hasta que el producto (software) se deje de usar.

- un producto puede tener muchos ciclos de vida de un proyecto en su ciclo de vida.

* Tipos Ciclo de Vida

- Secuencial
- Iterativo/Incremental
- Recursivo

* Administración de Proyectos dependiendo en ciclo de vida

La elección del ciclo de vida afecta a la administración que se debe realizar sobre el proyecto
cada modelo de proceso tiene diferentes características que pueden encajar mejor o peor según el proyecto.

Además según cual elijamos el desarrollo del proyecto la gestión nos será la misma

Procesos Empíricos vs Definidos

Proceso: Secuencia de pasos ejecutados para un propósito dado.

Proceso de software: Conjunto de actividades, métodos, prácticas que se usa para desarrollar o mantener software.

Factores determinantes

- procedimientos y métodos (definir actos y ciclos no y documentarlos)
- personas motivadas y con habilidades
- herramientas y equipo (materiales necesarios para llevar acabo)

Definidos

Asume que podemos repetir el mismo proceso

independientemente de todo y obtener los mismos resultados

• Hay una predictibilidad

• La experiencia es extrapolable

Empíricos (solo Ciclo de Vida iterativo)

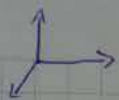
Asume que todo es cambiante y las variables son complejas.

- Inspección
- Adaptación
- Transparencia

? ciclos cortos

Exp. no extrapolable / Aprender del error

Triple restricción (enfoque tradicional)



Tradicional	Agil
Alcance fijo (se termina cuando se terminan)	Duración fija (time box)

variables de restricción $\rightarrow$ el balance de estas 3 afecta la calidad del proyecto

solo se podrán ~~ajustar~~ fijar 2 de las variables y la tercera se ajustará a las otras
si no se equilibran las 3 variables no hay calidad

Alcance: req. que se incluyan en el proyecto y las funcionalidades que tendrá el producto
en agil esto no es fijo ya que se va definiendo y refinando en base a la retroalimentación

Tiempo: se define en que fecha se compromete a finalizar el proyecto y entregar el producto
en agil el tiempo es fijo

Costo: el costo de todos los recursos implicados en el desarrollo del proyecto (equipamiento, infraestructura, equipos, salarios, etc.)

Lider de Proyecto:

realiza la toma de decisiones en su totalidad sin la inclusión de la opinión del equipo
asigna las tareas que debe realizar el equipo
maneja las relaciones con todos los stakeholders del proyecto

Equipo de Proyecto:

grupo de personas comprometidas con alcanzar un conj. de objetivos
tienen conocimiento, habilidades técnicas, motivación,
grupos pequeños usualmente
tienen un sentido de responsabilidad

Stakeholders:

interesados del proyecto (equipo de dirección, líder, equipo de proyecto, patrocinador)

Plan de Proyecto

artefacto que cumple la función de hoja de ruta de un proyecto,
se documentan todas las decisiones que se toman en el desarrollo.
(división de tareas, asignación de responsabilidad, equipo, calendario,
actividades de soporte $\rightarrow$ plan de contingencia, evaluar calidad, valor riesgo)

(objetivo y funciones específicas)

(que ciclo de vida van a implementar $\rightarrow$ cascada, iterativo, espiral)

(cuanto va a tardar / que tamaño va a tener / esfuerzo / costos)

(probabilidad $\cdot$ impacto)

(responsabilidades)

(que tarea específica con sus debilidades hay que desarrollar)

(definición clara de cada métrica en cada dominio (P, P, P))

(verificar que las métricas den bien)

- 1- Alcance del proyecto
- 2- Proceso y ciclo de vida
- 3- Estimación
- 4- Gestión de riesgos
- 5- Asignación de recursos
- 6- Programación de proyectos
- 7- Definición de métricas
- 8- Definición de controles